Ders 12 Genişletilmiş Notu: Mikroservislerin Dağıtımı ve Üretime Hazır Servisler

Ders 12 Genişletilmiş Notu: Mikroservislerin Dağıtımı ve Üretime Hazır Servisler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin biçimi
- 2. Mikroservislerin dağıtımı: üretim ortamı
- 3. Host başına birden çok servis örneği
- 4. Servisi sanal makine olarak dağıtmak
- 5. Servisi konteyner olarak dağıtmak
- 6. Sunucusuz (serverless) dağıtım
- 7. Hocanın değerlendirmesi: sanallaştırma seviyesinin yükselişi
- 8. Bölüm 11: Üretime hazır servisler
- 9. Güvenlik: monolitik uygulamada
- 10. Güvenlik: mikroservis mimarisinde
- 11. JWT ve OAuth 2.0
- 12. Hocanın ara özeti: rozet olarak erişim jetonu
- 13. Yapılandırılabilir servisler
- 14. Gözlemlenebilir servisler
- 15. Mikroservis gövdesi ve servis ağı
- 16. Hocanın kapanış değerlendirmesi

1. Dersin biçimi

Önceki derslerde olduğu gibi bu derste de Chris Richardson'ın "Microservices Patterns" kitabından öğrenci sunumları yapılır. Ders 8'de Bölüm 3 (süreçler arası iletişim) ve Bölüm 4 (saga ile transaction yönetimi) işlenmişti. Hoca derste, beşinci bölümden onuncu bölüme kadar olan kısımların (iş mantığı, olay kaynaklama, sorgular, dış API'ler, test) sunumlarının yapılamadığını, bunların kitaptan okunması gerektiğini söyler (00:02:45, 00:16:23).

Bu derste iki sunum vardır. Birincisinde bir öğrenci mikroservislerin dağıtımını (deployment) anlatır; slaytlar İngilizce, anlatım Türkçedir. İkincisinde başka bir öğrencinin önceden kaydettiği video izletilir; bu sunum kitabın 11. bölümü olan "Üretime hazır servisler geliştirme" konusunu işler (00:24:00). Hoca ilk sunumdan sonra dağıtım stratejilerini kendi anlatımıyla özetler, ikinci sunumun ortasında ve sonunda da kısa değerlendirmeler yapar.

> **Ek bilgi (derste anlatılmadı):** Kitapta dağıtım konusu büyük olasılıkla 12. bölüm ("Deploying microservices"), üretime hazırlık 11. bölümdür. Bu yüzden derste 11. bölümden önce 12. bölümün konusu sunulmuştur.

2. Mikroservislerin dağıtımı: üretim ortamı

Sunum, dört dağıtım yaklaşımını inceleyeceğini söyleyerek başlar: host başına birden çok servis örneği (Multiple Service Instances per Host), servisi sanal makine imajı olarak dağıtmak,

servisi konteyner olarak dağıtmak ve sunucusuz (serverless) dağıtım (00:06:01).

Neden bir dağıtım stratejisi gerekir? Bir uygulama, farklı dillerde ve çerçevelerde yazılmış onlarca servisten oluşabilir. Her servis küçük bir uygulama olduğu için üretimde yüzlerce uygulama bulunabilir. Sistem yöneticilerinin bu kadar sunucuyu ve servisi elle yapılandırması pratik değildir. Mikroservisleri ölçekli bir şekilde dağıtmak istiyorsanız otomatik bir dağıtım sürecine ihtiyacınız vardır (00:06:29).

2.1 Üretim ortamının dört yeteneği

Slayttaki şekil, üretim ortamının basitleştirilmiş görünümünü verir (ekrandan, 00:06:57). Şeklin solunda geliştirici, sağında kullanıcı vardır. Kullanıcı servisleri tüketir; istekleri bir yönlendirme (Routing) katmanından geçip A, B, C ve D servislerine gider. Geliştirici bir yandan izleme (Monitoring) bölümündeki panolar (Dashboards) ve uyarılar (Alerting) ile servisleri gözlemler ve sorun giderir, öte yandan servis yönetim arayüzü ile servisleri yapılandırıp yönetir. Dağıtım hattı (Deployment pipeline) da servisleri aynı arayüz üzerinden günceller. Ortamın altında çalışma zamanı servis yönetimi bileşeni bulunur.

Üretim ortamından istenen dört yetenek şunlardır (ekrandan, 00:08:17):

- **Servis yönetim arayüzü (service management interface):** Servis oluşturma, güncelleme ve yapılandırma olanağı sağlar. İdeal olarak bu, bir komut satırı ya da grafik arayüzün kullandığı bir API ile yapılır.
- **Çalışma zamanı servis yönetimi (runtime service management):** İstenen sayıda servis örneğinin çalıştığından emin olur. Bir servis çökerse ya da isteklere cevap veremez hâle gelirse üretim ortamı onu yeniden başlatır. Servisin çalıştığı makine çökerse servisler başka bir makinede yeniden başlatılmalıdır.
- **İzleme (monitoring):** Geliştiricilere servislerin ne yaptığına dair bilgi verir. Bir sorun varsa üretim ortamı geliştiricileri uyarmalı ve hatanın sebebini söylemelidir.
- **İstek yönlendirme (request routing):** Kullanıcıdan gelen istekleri servislere yönlendirir.

3. Host başına birden çok servis örneği

Sunuma göre en yaygın yaklaşım, makine başına birden çok servis çalıştırmaktır. Bu modelde geliştiriciler bir sanal ya da gerçek makinede birden çok servis çalıştırabilir (00:08:17).

3.1 Artıları

- Birden çok servis aynı sunucuyu ve işletim sistemini kullandığı için kaynak kullanımı görece verimlidir.
- Dağıtım görece hızlıdır, çünkü servisi bilgisayara kopyalayıp çalıştırmak yeterlidir. Java ile geliştirme yapılmışsa JAR ya da WAR dosyasını atmak, Ruby ya da Node.js ile geliştirme yapılmışsa kaynak kodunu kopyalamak yeterlidir (00:08:45).
- Servisin başlatılması için ek bir yük olmadığından servis diğer modellere göre daha hızlı başlar.

3.2 Eksileri

Slayttaki eksiler şunlardır (ekrandan, 00:10:00):

- Her örnek ayrı bir süreç olmadıkça servis örnekleri üzerinde kontrol azdır ya da hiç yoktur. Her örneğin kullandığı kaynağı sınırlamanın yolu yoktur. Sunumda verilen örnek: bir serviste bellek sızıntısı varsa sistemin bütün kaynaklarını tüketebilir; bu da diğer servisler için bir açlık (starvation) durumu yaratabilir (00:09:26).
- Birden çok servis aynı süreçte çalışıyorsa yalıtım (izolasyon) yoktur. Hatalı çalışan bir servis, aynı süreçteki diğer servislerin de kesintiye uğramasına yol açar.
- Dağıtımı yapan operasyon ekibinin servislerin en küçük ayrıntısını bilmesi gerektiği için dağıtım sırasında hata riski yüksektir. Bu yüzden geliştirme ekibiyle operasyon ekibi arasında bilgi alışverişi şarttır.

- Slaytta ayrıca şu madde vardır: aynı makinede birden çok servis örneği çalıştırmanın bir başka zorluğu, örneklerin makinelere yerleşimini belirlemektir. Her makinenin sabit bir kaynak kümesi (CPU, bellek vb.) vardır ve her örnek bir miktar kaynak ister; örnekleri makineleri aşırı yüklemeden verimli kullanacak şekilde atamak önemlidir.

Sunumda, sürüm yönetiminde de birçok sorunla karşılaşılabileceği söylenir (00:10:06).

4. Servisi sanal makine olarak dağıtmak

İkinci yöntem, servisi bir sanal makine imajı olarak paketleyip her servis örneğini ayrı bir sanal makine (VM) olarak dağıtmaktır (ekrandan, 00:10:20).

Slayttaki şekil, derleme zamanı (build time) ile çalışma zamanını (runtime) kesikli bir çizgiyle ayırır. Derleme zamanında servis kodu, dağıtım hattındaki VM imaj oluşturucuya (VM image builder) girer ve bir AMI (VM imajı) oluşturulur. Çalışma zamanında bu imaj, bir otomatik ölçekleme grubundaki (Autoscaling group) birden çok EC2 örneği olarak çalıştırılır; her EC2 örneğinde bir servis vardır. İstekler, Elastic Load Balancer üzerinden bu örneklere dağıtılır.

4.1 Artıları

- En önemli yararlardan biri, sanal makine imajının servisin bütün bağımlılıklarını içermesidir. Bu, bir servisin ihtiyaç duyduğu bağımlılıkların ayarlanması ve kurulması sırasındaki hataları ortadan kaldırır (00:10:26).
- Servis bir kez paketlendiğinde onu kapsayan bir kara kutu hâline gelir. Herhangi bir yerde değişiklik gerektirmeden dağıtılabilir. Böylece dağıtım çok daha kolay ve güvenilir olur.
- Servisler birbirinden tam olarak yalıtılmış biçimde çalışır.
- Servislere sabit bir CPU ve bellek kullanımı ayarlanabilir. Böylece bir servisin diğerlerinin kaynağını çalması engellenir.
- Amazon Web Services gibi teknolojilerle otomatik ölçekleme ve yük dengeleme yapılabilir (00:11:34).

4.2 Eksileri

- Kaynak kullanımı daha az verimlidir (00:11:51).
- Servis, kullanılıp kullanılmadığına bakılmaksızın ücretlendirildiği için maliyetler artabilir.
- Sanal makine imajı büyük olduğu için ağ üzerinden taşınması daha uzun sürer. İmajın oluşturulması da diğer yöntemlere göre daha uzun sürer. Servisin çalışmaya başlaması da işletim sisteminin açılmasını beklediği için daha uzun sürer (00:12:18).
- İşletim sisteminin sürüm ve yama ayarlamaları geliştiricilere kalır.

5. Servisi konteyner olarak dağıtmak

Üçüncü yöntem, servisi bir konteyner olarak dağıtmaktır. Slayttaki tanım şöyledir (ekrandan, 00:13:05): Konteynerler daha modern ve hafif bir dağıtım mekanizmasıdır. İşletim sistemi seviyesinde bir sanallaştırma mekanizmasıdırlar. Bir konteyner genellikle bir, bazen birden çok süreçten oluşur; bu süreçler onu diğer konteynerlerden yalıtan bir korumalı alanda (sandbox) çalışır. Örneğin bir Java servisi çalıştıran konteyner tipik olarak JVM sürecinden oluşur.

Konteynerde çalışan bir süreç açısından durum, kendi makinesinde çalışıyormuş gibidir. Tipik olarak kendi IP adresi vardır; bu da port çakışmalarını ortadan kaldırır. Örneğin bütün Java süreçleri 8080 portunu dinleyebilir. Her konteynerin kendi kök dosya sistemi de vardır. Konteyner çalışma ortamı, konteynerleri birbirinden yalıtmak için işletim sistemi mekanizmalarını kullanır.

5.1 Artıları

Konteyner yöntemi, sanal makine olarak dağıtmanın bütün avantajlarını taşır; ek olarak şunları sağlar (00:13:15):

- Sanal makinelere göre daha hafif olduğu için servislerin derlenmesi (build), ağ üzerinde taşınması ve başlatılması daha hızlıdır.

- Kaynak kullanımı servis başına CPU ve bellek olarak sınırlanabilir; bu da yine açlık durumunun oluşmasını engeller.
- Tam bir yalıtım sağlar. Servis örnekleri birbirinden yalıtılmış olduğu için bir serviste oluşan hata diğer servislerin bozulmasına yol açmaz.

5.2 Eksileri

İmajların yönetiminden geliştirici sorumludur. Amazon'un sunduğu gibi hazır konteyner çözümleri kullanılmıyorsa konteyner altyapısından ve onu yönetmekten de geliştiriciler sorumlu olur (00:14:09).

6. Sunucusuz (serverless) dağıtım

Dördüncü yöntem sunucusuz dağıtımdır. Slayttaki açıklama şöyledir (ekrandan, 00:14:30): Sunucusuz dağıtım teknolojisi mikroservis dağıtımı için bir başka stratejidir ve Java, Node.js ve Python servislerini destekler. AWS Lambda, dünyada geliştiricilerin yaygın kullandığı bir teknolojidir. Bu kalıpta servis bir ZIP dosyası olarak paketlenir ve durumsuz (stateless) bir servis olan Lambda fonksiyonuna yüklenir. Bir isteği işlerken çağrılacak fonksiyonun adını içeren üst veri (metadata) de verilebilir. Lambda fonksiyonu, istekleri karşılamaya yetecek kadar mikroservis örneğini otomatik olarak çalıştırır. Ücret, istek başına harcanan süreye ve tüketilen belleğe göre ödenir.

6.1 Artıları

- Amazon Web Services hizmetleriyle bütünleşik çalışır (00:14:54).
- Sistem yönetimi görevlerini geliştiricinin üzerinden alır.
- Servisleri ihtiyaç duyulduğu anda çalıştırır; ne kadar kapasiteye ihtiyaç duyulacağını tahmin etmekle uğraşılmaz.

6.2 Eksileri

- Özellikle Java geliştirmelerinde başlatma gecikmesi olabilir. Servisin anlık tepki vermesi isteniyorsa bu yöntem uygun olmayabilir (00:15:36).
- En fazla 300 saniyelik bir çalışma süresi vardır. Servis bu süreyi aşacaksa bu kalıp uygun olmayabilir.

  Ders sonrası not: Sunumda ve hocanın anlatımında AWS Lambda için en uzun çalışma süresi 300 saniye olarak verildi. Bu, kitabın yazıldığı dönemdeki sınırdı; büyük olasılıkla 2018 sonundan beri sınır 900 saniyedir (15 dakika).

7. Hocanın değerlendirmesi: sanallaştırma seviyesinin yükselişi

Hoca sunumdan sonra konuyu kendi anlatımıyla özetler (00:16:44). Elimizde artık derlenmiş ve test edilmiş mikroservisler vardır; bunları üretim ortamına taşımak gerekir. Bunun için çeşitli dağıtım, yani barındırma stratejileri vardır.

İlk akla gelen en basit yol, bütün servisleri tek makinede çalıştırmaktır. Hoca burada şunu sorar: o zaman monolitten ne farkı var? Pek bir farkı kalmaz; tek fark mimarinin mikroservis şeklinde kurgulanmış olmasıdır (00:17:02).

Sonraki adım, servisleri sanal makinelere taşımaktır: her ayrı sanal makine ayrı bir mikroservis gibi çalışır. Ancak sanal makinenin bir yükü vardır; aslında bir sunucu örneği çalıştırıyorsunuzdur ve onun ayağa kalkması bile bir derttir. Bir yük geldiğinde servisleri çoğaltmanız gerekirse sanal makinelerin çalışır hâle gelmesini beklemek zorundasınız. Üstüne maliyet vardır: kullanmasanız bile çalıştıracağınız için maliyet artar (00:17:12).

Bir sonraki adım sanallaştırmayı bir seviye yukarı, işletim sistemi seviyesine taşımaktır: konteyner teknolojisi. Hoca konteynerleri, işletim sistemini kendi içinde paket olarak barındıran ve daha hızlı çalışan mimari parçacıklar, yani sanal makinelerin hafifletilmiş hâlleri olarak tarif eder. Konteynerler tek makinede ya da bir kümede (cluster) çalışabilir; fiziksel yerleşimleri değiştirilebilir. Ama yine de orkestrasyon sorununu çözmek gerekir: herhangi bir anda hangi

konteynerler çalışacak, hangi servis konteynerinden kaç örnek çalışacak, işi bitenler nasıl kapatılıp kaynaklar geri alınacak? Hoca konteyner sanallaştırmasında orkestrasyonla ilgili çok ciddi sıkıntılar yaşanabileceğini söyler (00:18:24).

Son adım, sanallaştırmada bir kademe daha yukarı çıkmak, yani mikroservislerin kendisini sanallaştırmaktır. Her servis bir mikro konteyner gibi olur: yalnız gerektiğinde, istek geldiğinde ayağa kalkar, çalışır, sonra yeniden boşta bekleme (idle) durumuna döner. İşlemci gücü harcamadığı için de ona para ödenmez. Bunu tetikleyecek bir mekanizma gerekir; sunucusuz mimari (serverless architecture) budur. Ortada ne sanal makine ne de işletim sistemi taşıyan bir konteyner vardır; yalnız sizin servisiniz vardır. Servisi ya da onun çoğaltılmış örneklerini Lambda fonksiyonlarıyla çağırırsınız. Lambda fonksiyonu servisi "dürter": "İstek geldi, kalk, uyan, çalış" der; servis çalışır, sonucunu döndürür ve yeniden uyku ya da bekleme (standby) konumuna geçer (00:19:03).

> **Hocanın vurgusu:** Serverless'in önemli bir sakıncası 300 saniyelik süre sınırıdır; uzun süren servisler için Lambda fonksiyonları doğru cevap döndürmez. Ama hocaya göre serverless mimaride bu sınırı aşan servisleriniz varsa, büyük ihtimalle onlar mikroservis değildir; başka türlü bir hata yapmışsınızdır. Yapıyı gözden geçirip gerçekten mikro olan servisler yazmakta fayda vardır (00:20:11).

Hoca, bugün gelinen en son dağıtım sanallaştırma seviyesinin serverless olduğunu söyler: her servis uyuyan, dağıtıma hazır paketlenmiş çalıştırılabilir bir hizmet noktasıdır; hangisinin ne zaman ve kaç tane uyanacağı Lambda fonksiyonları üzerinden yapılan çağrılarla belirlenir. Bu da özellikle maliyet açısından, boş yere bir konteynerin ya da sanal sunucunun çalışmasını engeller (00:21:06).

8. Bölüm 11: Üretime hazır servisler

İkinci sunum kitabın 11. bölümünü işler. İçerik slaytına göre plan şöyledir (ekrandan, 00:24:11):

```
1. Giriş
2. Üretime Hazır Sistemler İçin Kalite Özellikleri
   - Güvenlik (Güvenli Hizmetler Geliştirme)
   - Yapılandırılabilirlik (Dışsallaştırılmış Yapılandırma Modelini Uygulama)
   - Gözlemlenebilirlik (Gözlemlenebilirlik Modelini Uygulama)
       i.   Sağlık Denetimi API'si
       ii.  Günlük (Log) Toplama
       iii. Dağıtık İzleme
       iv.  Kural Dışı Durum Takibi
       v.   Uygulama Metrikleri
       vi.  Denetim Günlükleme
3. Hizmetlerin Geliştirilmesini Basitleştirme - Mikro Hizmet Gövde Modeli
```

Bir servisin üretime sokulmaya hazır olması için kritik öneme sahip birkaç kalite özelliğini karşılaması gerekir: güvenlik, yapılandırılabilirlik ve gözlemlenebilirlik (00:24:30). Bölüm, önce mikroservis mimarisinde güvenliğin nasıl uygulanacağını, sonra yapılandırılabilir servislerin nasıl tasarlanacağını ve farklı yapılandırma mekanizmalarını ele alır; en sonda servisleri bir mikroservis gövdesi (microservice chassis) üzerinde geliştirerek bu kaygıların nasıl basitleştirilebileceğini gösterir.

9. Güvenlik: monolitik uygulamada

9.1 Neden önemli, ne değişir?

Bir şirketin bir veri ihlali yüzünden manşetlere çıkmasını istemiyorsanız güvenli uygulamalar geliştirmek çok önemlidir (00:25:33). Neyse ki mikroservis mimarisindeki güvenliğin çoğu yönü monolitik uygulamadakinden farklı değildir. Ancak mikroservis mimarisi, uygulama düzeyindeki güvenliğin bazı yönlerini farklı uygulamaya zorlar; örneğin kullanıcının kimliğini bir servisten diğerine geçirmek için bir mekanizma gerekir (00:26:00).

Güvenli yazılım geliştirmek için bir kuruluş çok çeşitli sorunlarla uğraşmalıdır: donanımın fiziksel güvenliği, aktarım hâlindeki ve durağan verinin şifrelenmesi, kimlik doğrulama ve yet-

kilendirme, yazılım açıklarını yamalama politikaları. Monolitik ya da mikroservis mimarisi kullanılsın, bu sorunların çoğu ortaktır. Bölüm, mikroservis mimarisinin uygulama düzeyindeki güvenliği nasıl etkilediğine odaklanır (00:26:32).

9.2 Güvenliğin dört yönü

Bir uygulama geliştiricisi öncelikle güvenliğin dört yönünden sorumludur (00:27:11):

- **Kimlik doğrulama (authentication):** Uygulamaya erişmeye çalışan uygulamanın ya da kişinin (principal) kimliğini doğrulamak.
- **Yetkilendirme (authorization):** Principal'ın belirtilen veri üzerinde istenen işlemi yapmaya yetkili olduğunu doğrulamak. Uygulamalar genellikle rol tabanlı güvenlik ve erişim kontrol listelerinin (ACL) bir bileşimini kullanır.
- **Denetim (auditing):** Güvenlik sorunlarını tespit etmek için bir principal'ın yaptığı işlemleri izlemek.
- **Güvenli süreçler arası iletişim:** Servislerin içindeki ve dışındaki bütün iletişim ideal olarak taşıma katmanı güvenliği (TLS) üzerinden olmalıdır. Servisler arası iletişimde bile kimlik doğrulama gerekebilir.

9.3 Monolitik FTGO uygulamasında güvenlik

Sunum, kitap boyunca kullanılan FTGO (Food to Go) yemek siparişi uygulamasını örnek alır. Kullanıcılar tüketici, kurye ve restoran personelidir; erişim web ve mobil tabanlıdır. FTGO Java ile yazılmıştır ve güvenlik çerçevesi olarak Spring Security kullanır (00:28:26).

İlk şekil istemcinin kimlik doğrulamasını gösterir (ekrandan, 00:29:35). Tüketici, kurye ya da restoran personeli bir tarayıcı ya da mobil uygulama üzerinden çalışır:

```
1. İstemci -> FTGO:  POST /login  (id=..., password=...)
   FTGO -> istemci:  HTTP/1.1 200 OK, Set-cookie: JSESSIONID=...
   (bir çerez olan oturum jetonunu elde etmek için oturum açılır)
2. İstemci -> FTGO:  GET /orders/order-xyz, Cookie: JSESSIONID=...
   (sonraki isteklerde kullanıcıyı tanımlayan oturum jetonu çerezi dahil edilir)
```

Yani kullanıcı adı ve şifreyle oturum açıldığında istemci kimlik bilgilerini içeren bir POST isteği yapar; FTGO bilgileri doğrular ve bir oturum jetonu (session token) döndürür; istemci sonraki her isteğe bu jetonu ekler. FTGO geleneksel bir Java EE uygulaması olduğu için oturum, bellek içi bir HTTP oturumudur; oturum jetonu `JSESSIONID` adlı bir HTTP çerezidir (00:29:20).

Güvenlik uygulamasının diğer önemli parçası güvenlik bağlamıdır (security context): o anki isteği yapan kullanıcı hakkındaki bilgiyi tutar. Spring Security, güvenlik bağlamını statik ve thread-local bir değişkende saklayan standart Java EE yaklaşımını kullanır; bu değişkene isteği işlemek için çağrılan her kod kolayca erişebilir. Bir istek işleyicisi, o anki kullanıcının kimliği ve rolleri gibi bilgileri almak için `SecurityContextHolder.getContext().getAuthentication()` çağrısını yapar (00:29:55).

İkinci şekil akışın ayrıntısını verir (ekrandan, 00:30:40):

```
1. Kullanıcı kullanıcı adı ve şifre ile giriş yapar:  POST /login (userId=jane&password=...)
2. Oturum Açma İşleyicisi (LoginHandler) veritabanından kullanıcı bilgilerini okur,
   oturumu başlatır (Oturum: userId: jane, roles: [CONSUMER])
3. İstemciye oturum çerezini döndürür:  HTTP/1.1 200 OK, Set-cookie: JSESSIONID=...
4. İstemci oturum çerezini sağlar:  GET /orders/order-xyz, Cookie: JSESSIONID=...
5. OturumTabanlıGüvenlikÖnleyicisi (SessionBasedSecurityInterceptor) oturumu okur
   ve güvenlik bağlamını kurar (userId: jane, roles: [CONSUMER])
6. SiparişAyrıntılarıİstekİşleyicisi (OrderDetailsRequestHandler) kullanıcı bilgisini
   güvenlik bağlamından okur
```

10. Güvenlik: mikroservis mimarisinde

10.1 Monolitin tasarımı neden kopyalanamaz?

Mikroservis mimarisi dağıtık bir mimaridir. Her dış istek, API ağ geçidi ve en az bir servis tarafından işlenir (00:31:23). Mikroservis uygulamasında güvenliği uygulamanın zorluğu, tasarımın monolitten kopyalanamamasıdır; çünkü monolitin güvenlik mimarisindeki iki unsur mikroservisler için yetersizdir (00:31:42):

- **Bellek içi güvenlik bağlamı:** Monolit kullanıcı kimliğini iletmek için thread-local gibi bir bellek içi bağlam kullanır. Servisler belleği paylaşamaz; bu yüzden kimliği bir servisten diğerine iletmek için farklı bir mekanizma gerekir.
- **Merkezi oturum:** Bellek içi güvenlik bağlamı anlamlı olmadığından bellek içi oturum da anlamlı değildir. Teoride birçok servis veritabanı tabanlı bir oturuma erişebilir; ancak bu, gevşek bağlılık (loose coupling) ilkesini ihlal eder. Mikroservis mimarisinde farklı bir oturum mekanizması gerekir (00:32:23).

10.2 API ağ geçidinde kimlik doğrulama

Mikroservisler için güvenlik unsurlarının birincisi, kimlik doğrulamanın API ağ geçidinde yapılmasıdır (ekrandan, 00:33:10). Şekilde iki tür istemci vardır:

```
API istemcileri (A):
A1. İstemci -> API ağ geçidi:  GET /orders/1, Authorization: ...CREDENTIALS...
    (kimlik bilgileri her istekte Authorization başlığında)
A2. Ağ geçidi -> Sipariş servisi:  GET /orders/1, ...SECURITY_TOKEN...

Oturum açma tabanlı istemciler (B):
B1. İstemci -> API ağ geçidi:  POST /login (id=..., password=...)
B2. Ağ geçidi -> istemci:  HTTP/1.1 200 OK  (güvenlik jetonu)
B3. İstemci -> ağ geçidi:  GET /orders/1, ...SECURITY_TOKEN...
    (her isteğe güvenlik jetonu eklenir)
B4. Ağ geçidi -> Sipariş servisi:  GET /orders/1, ...SECURITY_TOKEN...
```

API ağ geçidindeki kimlik doğrulama önleyicisi isteği doğrular; servislere, kullanıcıyı tanımlayabilmeleri ve yetkilendirebilmeleri için bir güvenlik jetonu iletir. Sunumda olaylar dizisi şöyle anlatılır (00:33:02): API istemcisi kimlik bilgilerini içeren bir istek yapar; ağ geçidi bilgileri doğrular, bir güvenlik jetonu oluşturur ve onu servis ya da servislere iletir. Oturum tabanlı istemci ise önce kimlik bilgilerini içeren bir oturum açma isteği yapar; ağ geçidi bir güvenlik jetonu döndürür; istemci işlem başlatan isteklerine bu jetonu ekler; ağ geçidi jetonu doğrulayıp servislere iletir.

10.3 Yetkilendirme

Güvenlik unsurlarının ikincisi yetkilendirmedir (00:33:54). İstemcinin kimliğini doğrulamak önemlidir ama yetersizdir. Uygulama, istemcinin istenen işlemi yapmaya yetkili olduğunu doğrulayan bir yetkilendirme mekanizması da uygulamalıdır. Örneğin FTGO'da `getOrderDetails()` sorgusunu yalnız siparişi veren tüketici ve o tüketiciye yardım eden bir müşteri hizmetleri temsilcisi çağırabilir (00:34:18).

Yetkilendirmenin uygulanabileceği iki yer vardır:

- **API ağ geçidi:** Bir kullanıcının belirli bir yola (path) erişmesine izin verilmiyorsa ağ geçidi isteği servise iletmeden önce reddedebilir.
- **Servisler:** Bir servis, URL'ler ve servis yöntemleri için rol tabanlı yetkilendirme uygulayabilir (00:34:47).

11. JWT ve OAuth 2.0

11.1 Kimlik ve rolleri JWT ile iletmek

Güvenlik unsurlarının üçüncüsü, kullanıcı kimliğini ve rollerini iletmek için JWT kullanmaktır (00:35:05). Mikroservis mimarisinde güvenliği uygularken API ağ geçidinin kullanıcı bilgisini

servislere iletmek için hangi tür jetonu kullanacağına karar vermek gerekir. Güvenlik servisine yapılan çağrıyı ortadan kaldıran bir yaklaşım, kullanıcı hakkında bilgi içeren şeffaf (transparent) bir jeton kullanmaktır. Bu jetonların en yaygını JSON Web Token'dır (JWT).

JWT, iki taraf arasında kullanıcı kimliği ve roller gibi talepleri (claims) güvenli biçimde göstermenin standart yoludur. Bir JWT'nin yükü (payload), kullanıcının kimliği ve rolleri gibi bilgileri ve son kullanma tarihi gibi diğer üst verileri içeren bir JSON nesnesidir. JWT, yalnız onu oluşturan tarafın (örneğin API ağ geçidi) ve alıcısının (örneğin bir servis) bildiği bir gizli anahtarla imzalanır. Bu sayede kötü niyetli bir üçüncü taraf JWT'yi taklit edemez ya da kurcalayamaz (00:35:42).

JWT'nin bir sorunu vardır (00:36:26): jeton kendi içinde bütün (self-contained) olduğu için geri alınamaz ve iptal edilemez. Servis, tasarım gereği JWT'nin imzasını ve son kullanma tarihini doğruladıktan sonra isteği işler. Sonuç olarak kötü niyetli birinin eline geçen bir JWT'yi iptal etmenin pratik bir yolu yoktur. Çözüm, JWT'leri kısa son kullanma süreleriyle yayınlamaktır. Kısa ömürlü JWT'lerin de bir dezavantajı vardır: oturumu aktif tutmak için uygulamanın JWT'leri sürekli yeniden yayınlaması gerekir. Neyse ki bu sorunu çözen birçok protokol vardır; OAuth 2.0 adlı güvenlik standardı bunlardan biridir (00:37:00).

> **Ek bilgi (derste anlatılmadı):** Bir JWT, nokta ile ayrılmış üç parçadan oluşur: başlık (header), yük (payload) ve imza (signature). İlk iki parça Base64URL ile kodlanmış JSON'dur; şifrelenmiş değildir. Bu yüzden imza jetonun değiştirilmediğini garanti eder ama içeriğini gizlemez.

11.2 API istemcisi için OAuth 2.0

İlk OAuth 2.0 şekli, API ağ geçidinin bir API istemcisinden gelen isteği nasıl doğruladığını gösterir (ekrandan, 00:37:30):

```
1. API istemcisi -> API ağ geçidi:  GET /orders/1, Authorization: Basic ...
2. Ağ geçidi -> Spring OAuth2 Kimlik Doğrulama Sunucusu:
   POST /oauth/token  userid=...&password=...   [şifre onayı isteği]
3. Sunucu -> ağ geçidi:  HTTP/1.1 200 OK  { "access_token": "AccessToken" ... }
4. Ağ geçidi -> Sipariş servisi:  GET /orders/1, Authorization: Bearer ...
   (jeton kullanıcı kimliğini ve rollerini içerir)
```

Sunumdaki anlatımla: istemci, kimlik bilgilerini temel kimlik doğrulama (Basic authentication) ile vererek bir istek yapar. Ağ geçidi, OAuth 2.0 kimlik doğrulama sunucusuna bir şifre onayı (Password Grant) isteği yapar. Kimlik doğrulama sunucusu istemcinin kimlik bilgilerini doğrular ve bir erişim jetonu (access token) ile bir yenileme jetonu (refresh token) döndürür. Ağ geçidi, servislere yaptığı isteklere erişim jetonunu ekler; servis jetonu doğrular ve isteği yetkilendirmek için kullanır (00:37:37).

11.3 Oturum tabanlı istemci için OAuth 2.0

İkinci şekil, ağ geçidinin oturum tabanlı istemciler için OAuth 2.0'ı nasıl kullandığını gösterir (ekrandan, 00:38:46):

```
1. Oturum açma tabanlı istemci -> ağ geçidinin oturum açma işleyicisi:
   POST /login  userId=...&password=...
2. Oturum açma işleyicisi -> Spring OAuth2 Kimlik Doğrulama Sunucusu:
   POST /oauth/token  userid=...&password=...   [şifre onayı isteği]
3. Sunucu -> işleyici:  HTTP/1.1 200 OK  { "access_token": "AccessToken" ... }
   (erişim jetonu ve yenileme jetonu)
4. Ağ geçidi -> istemci:  HTTP/1.1 200 OK
   Set-Cookie: access_token=...   Set-Cookie: refresh_token=...
5. İstemci -> ağ geçidi:  GET /orders/1, Cookie: access_token=...; refresh_token=...
6. Oturum kimlik doğrulama önleyicisi erişim jetonunu doğrular -> Sipariş servisi:
   GET /orders/1, Authorization: Bearer AccessToken
```

Yani istemci kimlik bilgilerini ağ geçidine POST eder; ağ geçidinin oturum açma işleyicisi OAuth 2.0 sunucusuna şifre onayı isteği yapar; sunucu bilgileri doğrulayıp erişim ve yenileme

jetonlarını döndürür; ağ geçidi bu jetonları istemciye örneğin çerez olarak verir; istemci sonraki isteklerine jetonları ekler; ağ geçidinin oturum kimlik doğrulama önleyicisi erişim jetonunu doğrular ve servislere yapılan isteklere ekler (00:38:44).

12. Hocanın ara özeti: rozet olarak erişim jetonu

Hoca sunumun uzun ve ayrıntılı olduğunu söyleyip araya girer ve güvenlik kısmını toparlar (00:40:01). Önce monolitik mimaride güvenliğin nasıl sağlandığı, yani kimlik doğrulama (authentication) ve yetkilendirme (authorization) mekanizmaları anlatıldı. Sonra bunu bir mikroservis ortamına taşıdığımızda aynı şekilde çalışamayacağımız, farklı bir erişim jetonuna (access token) ihtiyaç olduğu söylendi; bunun için de OAuth 2.0 mekanizmasının ne olduğu ve nasıl çalıştığı ana hatlarıyla aktarıldı.

Hocanın benzetmesiyle: bir servise erişip ondan hizmet almak için elimizde bir jeton, bir rozet (badge) ya da kart olması gerekir. Bu rozet "gerçekten ben buyum ve bu hizmete erişmeye yetkiliyim" der. Sorun bu rozetin, yani kimlik doğrulama jetonunun nasıl elde edileceğidir (00:40:38).

13. Yapılandırılabilir servisler

13.1 Dışsallaştırılmış yapılandırma

Bir servis farklı ortamlarda farklı yapılandırma değerleriyle çalışır. Sunumda, farklı yapılandırma özellik kümelerini kaynak koduna gömüp çalışma zamanında uygun kümeyi seçmenin söz konusu olduğu, ama dışsallaştırılmış yapılandırma (externalized configuration) modeliyle uygun yapılandırma özelliklerinin servise çalışma zamanında dışarıdan sağlanabileceği söylenir (00:41:15).

İki ana yaklaşım vardır: itme (push) ve çekme (pull) modelleri (00:41:33).

13.2 İtme modeli

İtme modelinde dağıtım altyapısı, yapılandırma özelliklerini servis örneğine (service instance) örneğin işletim sistemi ortam değişkenleri ya da bir yapılandırma dosyasıyla aktarır (00:41:38). Model, dağıtım ortamının ve servisin iş birliğine dayanır: dağıtım ortamı bir servis örneği oluştururken yapılandırma özelliklerini sağlar; servis örneği başlatıldığında bunları okur (00:42:03).

İtme modeli bir servisi yapılandırmak için etkili ve yaygın bir mekanizmadır. Bununla birlikte iki sınırı vardır (00:42:24):

- Çalışan bir servisi yeniden yapılandırmak imkânsız değilse de zordur. Örneğin çalışan bir sürecin ortam değişkenlerini değiştiremezsiniz.
- Yapılandırma değerlerinin birçok servisin tanımına dağılma riski vardır.

Bu sınırlar yüzünden çekme tabanlı bir model düşünülebilir.

13.3 Çekme modeli

Çekme modelinde servis örneği yapılandırma özelliklerini bir yapılandırma sunucusundan okur (ekrandan, 00:43:24). Şekilde dağıtım altyapısı, Sipariş Geçmişi Hizmeti örneğini oluşturur ve ona yalnız ortam değişkeni olarak yapılandırma sunucusunun adresini (`CONFIG_SERVER_URL=...`) verir. Servis örneği açılışta (startup) `getConfiguration("orderHistoryService")` çağrısıyla yapılandırma sunucusundan asıl değerleri alır:

```
BOOTSTRAP_SERVERS=kafka1:9092
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=...
```

Yapılandırma sunucusu çeşitli yollarla gerçeklenebilir (00:43:41): Git gibi bir sürüm kontrol sistemi, SQL ve NoSQL veritabanları ya da özel yapılandırma sunucuları. Özel sunuculara

örnek olarak Spring Cloud Config Server, kimlik bilgileri gibi hassas veriler için bir depo olan HashiCorp Vault ve AWS Parameter Store verilir.

Yapılandırma sunucusu kullanmanın artıları (00:44:03):

- **Merkezi yapılandırma:** Bütün yapılandırma özellikleri tek yerde tutulur; bu da yönetimi kolaylaştırır.
- **Hassas verinin şifresinin çözülmesi:** Veritabanı kimlik bilgileri gibi hassas verileri şifrelemek iyi bir güvenlik uygulamasıdır. Şifrelemenin bir zorluğu, servis örneğinin şifreyi çözmesi gerekmesi, dolayısıyla şifreleme anahtarlarına ihtiyaç duymasıdır. Bazı yapılandırma sunucuları özelliklerin şifresini servise döndürmeden önce otomatik çözer.
- **Dinamik yeniden yapılandırma:** Bir servis örneği, yoklama (polling) yoluyla güncellenmiş özellik değerlerini algılayıp kendini yeniden yapılandırabilir.

Başlıca eksisi, altyapı tarafından sağlanmadığı sürece kurulması ve bakımı yapılması gereken ek bir altyapı parçası olmasıdır. Neyse ki Spring Cloud Config gibi, bir yapılandırma sunucusu çalıştırmayı kolaylaştıran açık kaynak çerçeveler vardır (00:45:05).

14. Gözlemlenebilir servisler

14.1 Altı gözlemlenebilirlik modeli

Bir servis geliştiricisi olarak, servisin daha kolay yönetilmesi, sorunlarının giderilmesi ve davranışının anlaşılması için uygulanması gereken birkaç model vardır (00:45:33):

- **Sağlık denetimi API'si (health check API):** Servisin sağlığını döndüren bir uç nokta (endpoint) açılır.
- **Günlük toplama (log aggregation):** Servis etkinliği günlüğe kaydedilir; günlükler arama ve uyarı sağlayan merkezi bir günlük sunucusuna yazılır.
- **Dağıtık izleme (distributed tracing):** Her dış isteğe benzersiz bir kimlik atanır; istekler servisler arasında akarken izlenir.
- **İstisna izleme (exception tracking):** İstisnalar, onları tekilleştiren (deduplicate), geliştiricileri uyaran ve her istisnanın çözümünü izleyen bir istisna izleme servisine bildirilir.
- **Uygulama metrikleri (application metrics):** Servisler sayaç ve gösterge gibi metrikleri tutar ve bir metrik sunucusuna sunar.
- **Denetim günlüğü (audit logging):** Kullanıcı eylemleri günlüğe kaydedilir.

Slayttaki şekil 11.9 (ekrandan, 00:46:55), ortada "gözlemlenebilir" bir servisi ve onun etrafındaki parçaları gösterir. Servisin içinde sağlık denetimi, dağıtık izleme aktarıcısı, metrik aktarıcısı, denetim veritabanı adaptörü, istisna raportörü ve günlük adaptörü vardır. Bunlar sırasıyla sağlık denetimi başlatıcısına (izleme servisi gibi), dağıtık izleme sunucusuna, metrik servisine, denetim veritabanına, istisna izleme servisine ve günlük dosyası üzerinden günlük toplama hattına bağlanır. Şeklin anahtarı, servis içindeki parçaların geliştiricinin, dış altyapının ise operasyonun sorumluluğunda olduğunu gösterir: geliştiriciler servislerinin gözlemlenebilir olmasını sağlamaktan, operasyon da servislerin açığa çıkardığı bilgiyi toplayan altyapıdan sorumludur (00:47:13).

14.2 Sağlık denetimi API'si

Bazen bir servis çalışıyordur ama istekleri işleyemiyordur (00:47:32). Örneğin yeni başlatılmış bir servis örneği istek kabul etmeye henüz hazır olmayabilir. Ya da bir servis örneği sonlanmadan başarısız olabilir: örneğin bir hata (bug), Tüketici Servisi'nin bir örneğinin veritabanı bağlantılarının tükenmesine ve veritabanına erişememesine yol açabilir.

Bir servis örneğinin, istekleri işleyip işleyemeyeceğini dağıtım altyapısına söyleyebilmesi gerekir. İyi bir çözüm, servisin bir sağlık denetimi uç noktası uygulamasıdır (ekrandan, 00:48:24). Şekilde sağlık denetimi çağırıcısı (invoker; örneğin izleme sistemi, servis kayıt defteri ve diğerleri) servisin sağlık denetimi uç noktasını periyodik olarak çağırır. Sağlık denetimi istek işleyicisi, servisin altyapı servisleriyle olan bağlantılarını test eder: mesajlaşma adaptörü üzerinden mesaj aracısını, veritabanı adaptörü üzerinden MySQL'i denetler.

Sağlık denetimlerinde iki mesele vardır (00:48:54): birincisi, servis örneğinin sağlığını bildiren uç noktanın gerçeklenmesi; ikincisi, dağıtım altyapısının bu uç noktayı çağıracak şekilde

yapılandırılması.

14.3 Günlük toplama

Günlükler değerli bir sorun giderme aracıdır. Uygulamada neyin yanlış gittiğini bilmek istiyorsanız günlük dosyaları iyi bir başlangıç noktasıdır. Ancak mikroservis mimarisinde günlükleri kullanmak zordur, çünkü günlükler birçok servise dağılmıştır (00:49:21). Çözüm, bütün servislerin günlüklerini arama ve uyarıyı destekleyen merkezi bir yerde toplamaktır.

Slayttaki şekil 11.11'de (ekrandan, 00:50:00) Hizmet A örnek 1, Hizmet B örnek 1 ve Hizmet A örnek 2'nin her biri bir günlük kütüphanesiyle kendi günlüğünü yazar. Bu günlükler bir günlük hattına (logging pipeline) girer ve hat onları günlük sunucusuna gönderir. Kullanıcı günlükleri görüntüler, sunucu da kullanıcıya bildirim yapar. Günlük hattı ve sunucusu genellikle operasyonun sorumluluğundadır; ama servis geliştiricileri yararlı günlükler üreten servisler yazmaktan sorumludur (00:50:03).

Bir servisin günlük üretmesinde iki husus vardır (00:50:41):

- **Hangi günlük kütüphanesi?** Çoğu dilde doğru yapılandırılmış günlük kaydı üretmeyi kolaylaştıran kütüphaneler vardır. Java'da üç yaygın kütüphane: Logback, Log4j ve JUL (java.util.logging).
- **Günlük nereye yazılacak?** Günlük çerçevesi, dosya sisteminde iyi bilinen bir konumdaki bir günlük dosyasına yazacak şekilde yapılandırılır.

Günlük toplama altyapısı günlükleri toplamaktan, depolamaktan ve kullanıcının aramasını sağlamaktan sorumludur (00:51:19). Yaygın bir altyapı ELK yığınıdır; üç açık kaynak üründen oluşur:

- **Elasticsearch:** Günlük sunucusu olarak kullanılan, metin arama odaklı bir NoSQL veritabanı.
- **Logstash:** Servis günlüklerini toplayıp Elasticsearch'e yazan bir günlük hattı.
- **Kibana:** Elasticsearch için bir görselleştirme aracı.

Diğer açık kaynak günlük hatları arasında Fluentd ve Apache Flume vardır. Günlük sunucusu olarak AWS CloudWatch Logs gibi bulut hizmetleri ve çok sayıda ticari ürün de bulunur (00:51:51).

14.4 Dağıtık izleme

Uygulamanın ne yaptığını anlamanın iyi bir yolu dağıtık izlemedir (00:52:10). Dağıtık izleme, monolitik bir uygulamadaki performans profil çıkarıcısına (profiler) benzer: bir istek işlenirken yapılan servis çağrılarının ağacı hakkında bilgi (başlangıç ve bitiş zamanı gibi) kaydeder. Böylece bir dış isteğin işlenmesi sırasında servislerin nasıl etkileştiği ve zamanın nerede harcandığı görülebilir.

Terminoloji (00:53:14):

- **İz (trace):** Bir dış isteği temsil eder; bir ya da daha fazla yayılımdan oluşur.
- **Yayılım (span):** Bir işlemi temsil eder. Temel nitelikleri işlem adı, başlangıç zaman damgası ve bitiş zamanıdır. Bir yayılım, iç içe işlemleri temsil eden bir ya da daha fazla alt yayılıma (child span) sahip olabilir. Örneğin üst seviye bir yayılım API ağ geçidinin çağrılmasını, onun alt yayılımları da ağ geçidinin servisleri çağırmasını temsil edebilir. Toplanan ayrıntı düzeyine göre bir yayılım, bir servis içindeki bir işlemin çağrılmasını da temsil edebilir.

Slayttaki Zipkin ekranı (ekrandan, 00:53:16) bu yapıyı somutlaştırır. Gösterilen iz, FTGO API ağ geçidine gelen ve Sipariş Servisi'ne yönlendirilen bir isteğe aittir. Ekranda iki servis, derinlik 2 ve toplam 2 yayılım görünür. Üst yayılım `ftgo-api-gateway` üzerindeki yaklaşık 18,9 ms süren `get` işlemidir; alt yayılım `ftgo-order-service` üzerindeki yaklaşık 12,7 ms süren `get /orders/{orderId}` işlemidir. İkisi birlikte izi oluşturur.

Dağıtık izlemenin iki parçası vardır (00:54:32):

- **Araç kütüphanesi (instrumentation library):** Her servis tarafından kullanılır. İzleri ve yayılımları yönetir; giden isteklere güncel iz kimliği ve üst yayılım kimliği gibi izleme bilgilerini ekler; izleri dağıtık izleme sunucusuna bildirir.
- **Dağıtık izleme sunucusu:** Yayılımları birleştirip tam izleri oluşturur, veritabanına kaydeder ve görselleştirmek için bir arayüz sağlar. En yaygın örnek Open Zipkin'dir (00:56:04).

Şekil 11.13 (ekrandan, 00:55:12) mekanizmayı gösterir:

```
İstek: GET /orders/1 HTTP/1.1  ->  API ağ geçidi (araç kütüphanesi)
API ağ geçidi -> Sipariş Hizmeti:
    GET /orders/1 HTTP/1.1
    X-B3-TraceId: XYZ          (İzKimliği)
    X-B3-ParentSpanId: ABC     (ÜstYayılımKimliği)

Dağıtık izleme sunucusuna aktarılanlar:
    Hizmet: API ağ geçidi     İzKimliği: XYZ   ÜstYayılım: YOK
    Hizmet: Sipariş Hizmeti   İzKimliği: XYZ   ÜstYayılım: ABC   Yayılım: DEF

Sunucu -> iz veritabanı; kullanıcı izleri görüntüler.
(ABC yayılımı: API ağ geçidi;  DEF yayılımı: Sipariş Hizmeti)
```

Servis kodu araç kütüphanesini doğrudan çağırabilir; ancak bu, izleme mantığını (instrumentation logic) iş mantığıyla ve diğer mantıklarla iç içe geçirir. Daha temiz bir yaklaşım, önleyiciler (interceptors) ya da yön odaklı programlama (aspect-oriented programming, AOP) kullanmaktır (00:55:32).

14.5 Uygulama metrikleri

Üretim ortamının önemli bir parçası gözetim (monitoring) ve uyarıdır (00:56:10). Gözetim sistemi, teknoloji yığınının her katmanından uygulamanın sağlığı hakkında kritik bilgi veren metrikleri toplar. Metrikler, CPU, bellek ve disk kullanımı gibi altyapı düzeyindeki metriklerden, servis isteği gecikmesi ve yürütülen istek sayısı gibi uygulama düzeyindeki metriklere kadar uzanır. Örneğin Sipariş Servisi verilen, onaylanan ve reddedilen sipariş sayılarıyla ilgili metrikleri toplar. Metrikler, görselleştirme ve uyarı sağlayan bir metrik servisinde toplanır.

Metrikler periyodik olarak örneklenir. Her metrik örneğinin üç özelliği vardır (00:56:53):

- **Ad:** metriğin adı.
- **Değer:** sayısal değer.
- **Zaman damgası:** örneklemenin zamanı.

Gözetimin birçok yönü operasyonun sorumluluğundadır; ama servis geliştiricisi metriklerin iki yönünden sorumludur (00:57:09): birincisi, servisini kendi davranışıyla ilgili metrikleri toplayacak şekilde donatmak; ikincisi, bu servis metriklerini JVM ve uygulama çerçevesinden gelen metriklerle birlikte metrik sunucusuna sunmak.

14.6 İstisna izleme

Bir servis nadiren bir istisnayı günlüğe kaydeder; kaydettiğinde de temel nedeni bulmak önemlidir (00:57:57). İstisna bir arızanın ya da bir programlama hatasının belirtisi olabilir. İstisnaları görmenin geleneksel yolu günlük kayıtlarına bakmaktır; günlük dosyasında bir istisna göründüğünde günlük sunucusu uyarı verecek şekilde de yapılandırılabilir.

Daha iyi bir yol, servisin istisnaları örneğin bir REST API üzerinden bir istisna izleme servisine bildirmesidir (ekrandan, 00:59:22). Şekil 11.15'te Sipariş Hizmeti içindeki istisna izleyen istemci kütüphanesi istisnayı `POST /exceptions` ile bildirir; gövdede bir yığın izi vardır:

```
POST /exceptions

java.lang.NullPointerException
    at net.chrisrichardson.ftgo...
    at net.chrisrichardson.ftgo...
    at net.chrisrichardson.ftgo...
```

İstisna izleme servisi istisnaları istisna veritabanına kaydeder, tekilleştirir, uyarı üretir ve istisnaların çözümünü yönetir; kullanıcı istisnaları bir arayüzden görüntüler ve yönetir. Birçok istisna izleme servisi vardır; bazıları, örneğin Honeybadger, tamamen bulut tabanlıdır. Bu servisler genellikle çeşitli diller için istemci kütüphaneleri sağlar (00:58:55).

14.7 Denetim günlüğü

Denetim günlüğünün amacı her kullanıcının eylemlerini kaydetmektir (ekrandan, 01:00:00). Bir denetim günlüğü genellikle uyumluluğun (compliance) sağlanması, şüpheli davranışın tespit edilmesi ve müşteri hizmetlerine yardımcı olmak için kullanılır. Her denetim kaydı kullanıcının kimliğini, yaptığı eylemi ve iş nesnesini ya da nesnelerini (business objects) kaydeder. Uygulama denetim günlüğünü genellikle bir veritabanı tablosunda tutar.

Denetim günlüğünü uygulamanın birkaç yolu vardır (01:00:20):

- İş mantığına denetim günlüğü kodu eklemek.
- Yön odaklı programlama (AOP) kullanmak.
- Olay kaynaklama (event sourcing) kullanmak.

  Ek bilgi (derste anlatılmadı): Olay kaynaklamada bir nesnenin durumu, onu değiştiren olayların sırayla saklanmasıyla tutulur. Her olay zaten "kim, neyi, ne zaman değiştirdi" bilgisini taşıyabildiği için olay deposu doğal bir denetim günlüğü işlevi görür. Kitapta olay kaynaklama, derste sunulamayan bölümlerden birinde anlatılır.

15. Mikroservis gövdesi ve servis ağı

15.1 Mikroservis gövdesi

Bölümün son konusu servis geliştirmeyi basitleştirmektir (01:00:33). Servis geliştirmenin çok daha hızlı bir yolu, servisleri bir mikroservis gövdesi (microservice chassis) üzerinde kurmaktır. Mikroservis gövdesi; istisna izleme, günlükleme, sağlık denetimi, dışsallaştırılmış yapılandırma ve dağıtık izleme gibi kaygıları ele alan bir çerçeve ya da çerçeveler topluluğudur. Gövde kullanıldığında bu kaygılar için çok az kod yazılır (01:00:58).

Sunumda gövdenin ele aldığı kaygılar şöyle sayılır (01:01:39): dışsallaştırılmış yapılandırma, sağlık denetimleri, uygulama metrikleri, servis keşfi, devre kesiciler ve dağıtık izleme. Gövde yazılması gereken kod miktarını önemli ölçüde azaltır; hiç kod yazmak gerekmeyebilir. Geliştirici gövdeyi kendi gereksinimlerine göre yapılandırır ve servisin iş mantığına odaklanır.

FTGO uygulaması mikroservis gövdesi olarak Spring Boot ve Spring Cloud kullanır. Spring Boot dışsallaştırılmış yapılandırma gibi işlevler, Spring Cloud ise devre kesiciler gibi işlevler sağlar. FTGO, servis keşfi için altyapıya dayanmakla birlikte istemci taraflı servis keşfini (client-side service discovery) de uygular (01:02:12).

15.2 Gövdeden servis ağına

Mikroservis gövdesi, devre kesiciler gibi önemli kaygıları uygulamanın iyi bir yoludur. Ancak bir engeli vardır: kullandığınız her programlama dili için ayrı bir gövde gerekir (ekrandan, 01:03:34). Slayttaki örnekle: Java/Spring geliştiricisiyseniz Spring Boot ve Spring Cloud kullanışlıdır, ama Node.js tabanlı bir servis yazmak istiyorsanız bunların hiçbir yardımı olmaz.

Bu sorunu önleyen ve yeni ortaya çıkan bir alternatif, bu işlevselliğin bir kısmını servisin dışında, servis ağı (service mesh) denen yapıda uygulamaktır. Servis ağı, bir servis ile diğer servisler ve dış uygulamalar arasındaki iletişime aracılık eden ağ altyapısıdır (01:03:18). Bir servise giren ve çıkan bütün ağ trafiği servis ağından geçer. Servis ağı devre kesiciler, dağıtık izleme, servis keşfi, yük dengeleme ve kural tabanlı trafik yönlendirme gibi kaygıları uygular; ayrıca servisler arasında TLS tabanlı IPC ile süreçler arası iletişimi güvence altına alır.

Slayttaki şekil 11.17'de API ağ geçidi, Sipariş Servisi ve Restoran Servisi'nin her birinde bir mikroservis gövdesi vardır; ama bu gövdeler artık yalnız günlükleme, uygulama metrikleri, dışsal yapılandırma, sağlık denetimi ve dağıtık izleme gibi daha az işlevi içerir. Devre kesici, dağıtık izleme, servis keşfi, akıllı trafik yönlendirme, yük dengeleme ve güvenli iletişim, servislerin altındaki servis ağı katmanına taşınmıştır; onun altında da dağıtım altyapısı bulunur.

Sonuç olarak bu kaygıları servislerde uygulamaya gerek kalmaz; mikroservis gövdesi çok daha basit olur ve yalnız uygulama koduyla sıkı bütünleşik kaygıları (dışsallaştırılmış yapılandırma, sağlık denetimi gibi) uygular (01:03:58).

> **Ek bilgi (derste anlatılmadı):** Servis ağının yaygın örnekleri Istio ve Linkerd'dir. Bu araçlar genellikle her servis örneğinin yanına bir vekil (sidecar proxy, örneğin Envoy) yerleştirir ve servisin bütün ağ trafiğini bu vekil üzerinden geçirir.

16. Hocanın kapanış değerlendirmesi

Hoca, ikinci sunumun kapsamlı bir konuyu bütünüyle aldığını söyler (01:04:52). Özetle anlatılanlar, bir mikroservis mimari ortamının çalışabilmesi için gereken destek hizmetleridir: güvenlikten kayıtlara, olağan dışı durumların ele alınmasına kadar pek çok noktada ne yapılabileceği, bu hizmetlerin nerede barındırılabileceği ve başka nasıl çözülebileceği.

> **Hocanın vurgusu:** Bir mikroservis ortamının çalışmasını ve sistemin destek etkinliklerinin düzgün yürümesini sağlamak için, doğal olarak monolitik yapıdan farklı çeşitli çözümler uyarlamak gerekir. Sunum bunların yolunu yordamını ve Spring ile nasıl yapılabileceğini özetlemiştir (01:05:33).

Dersin Toparlaması

Bu derste mikroservislerin üretime nasıl taşındığı iki yönden ele alındı. İlk yön dağıtımdı: servisleri aynı makinede çalıştırmaktan sanal makinelere, oradan konteynerlere ve sonunda sunucusuz dağıtıma uzanan bir çizgide her adım bir öncekinin sorununu çözer. Tek makine verimli ama yalıtımsızdır; sanal makine yalıtım getirir ama ağır ve pahalıdır; konteyner hafiftir ama orkestrasyon ister; serverless'te yalnız servis kalır, ama süre sınırı ve başlatma gecikmesi vardır. Hocanın uyarısıyla, süre sınırına takılan bir servis büyük ihtimalle gerçekten mikro değildir.

İkinci yön, bir servisin üretime hazır olması için taşıması gereken kalite özellikleriydi. Güvenlikte monolitin bellek içi oturumu ve thread-local güvenlik bağlamı mikroservislerde işe yaramaz; kimlik doğrulama API ağ geçidinde yapılır ve kimlik, JWT gibi imzalı jetonlarla servislere taşınır; OAuth 2.0 erişim ve yenileme jetonlarıyla kısa ömürlü jeton sorununu çözer. Yapılandırma, itme ya da çekme modeliyle servisin dışına alınır. Gözlemlenebilirlik için sağlık denetimi, günlük toplama, dağıtık izleme, istisna izleme, metrikler ve denetim günlüğü kullanılır. Bu kaygıların tamamını her serviste yeniden yazmamak için mikroservis gövdesi, dil bağımlılığını azaltmak için de servis ağı kullanılır. Hoca, kitapta sunulamayan bölümlerin (iş mantığı, olay kaynaklama, sorgular, dış API'ler, test) okunmasını ister; sonraki derste monolitin mikroservislere dönüştürülmesi (refactoring) konusunun işlenmesi beklenir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.